

Section 79 — Directed Graph Decomposition

From cycles to a DAG

This lesson derives Problems A–C. Problem D is a fresh transfer problem whose derivation appears only in the editorial.

Problem A: strongly connected components

Mutual reachability is an equivalence relation. Contracting each equivalence class removes every directed cycle between classes, so the condensation is a DAG. Kosaraju orders vertices by completed DFS time, then explores the reversed graph in decreasing completion order. The key boundary fact is that the first still-unassigned vertex cannot be entered from another unassigned source component in the reversed condensation.

Once component IDs are known, scan original edges and mark inter-component indegrees and outdegrees. Parallel condensation edges do not change zero/nonzero status.

Problem B: 2-SAT as SCC contradiction

Clause $(a \text{ OR } b)$ is equivalent to implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. An assignment is impossible exactly when a literal and its negation lie in the same SCC: each would force the other. Conversely, processing SCCs in reverse condensation order lets the first member of each unassigned complementary pair be set false, producing a consistent assignment.

Problem C: directed Euler trails

An open Euler trail has one vertex with $\text{out}=\text{in}+1$, one with $\text{in}=\text{out}+1$, and balanced degrees elsewhere. A circuit balances every vertex. These conditions are necessary but not sufficient: all used edges must also belong to the traversal reachable from the requested start.

Hierholzer follows unused edges until stuck and appends vertices while backtracking. Each edge is removed once. Sorting outgoing edges in reverse lets the smallest available endpoint be popped in constant time; the final length check detects disconnected edge sets.

Recognition and proof habits

- Mutual reachability suggests SCC contraction.
- Boolean implications suggest a graph on literals.
- “Use every edge once” suggests Euler, not Hamilton.
- “Every path from a root” suggests dominators, the editorial-only transfer.

Worked traces and implementation

If $1 \leftrightarrow 2$, $3 \leftrightarrow 4$, and edges go $2 \rightarrow 3$, $4 \rightarrow 5$, the SCCs are $\{1,2\}$, $\{3,4\}$, and $\{5\}$. Their condensation is a path. A cycle among contracted components would make all its vertices mutually reachable and therefore one SCC. Kosaraju’s finishing order should be justified on this DAG, not memorized as an unexplained reversal.

For clause $(x \text{ OR } \text{not } y)$, add both implications $\text{not } x \rightarrow \text{not } y$ and $y \rightarrow x$. Verify the final component-order assignment on unit clauses (x) and $(\text{not } x)$, because the comparison sign depends on component numbering.

On edges $1 \rightarrow 2$, $2 \rightarrow 1$, $1 \rightarrow 3$, Hierholzer closes a cycle before the final edge. Appending vertices only when stuck and reversing the result splices it correctly. Degree conditions choose endpoints; length $m+1$ proves all edges were reached.

Use iterative DFS on deep graphs and explicit exit actions for finishing order. Store edge IDs for parallel Euler edges. On tiny inputs, use transitive closure, enumerate Boolean assignments, and backtrack over unused edge IDs.

Exercises

A–C correspond to the three derivations. D deliberately supplies no lesson recipe.